

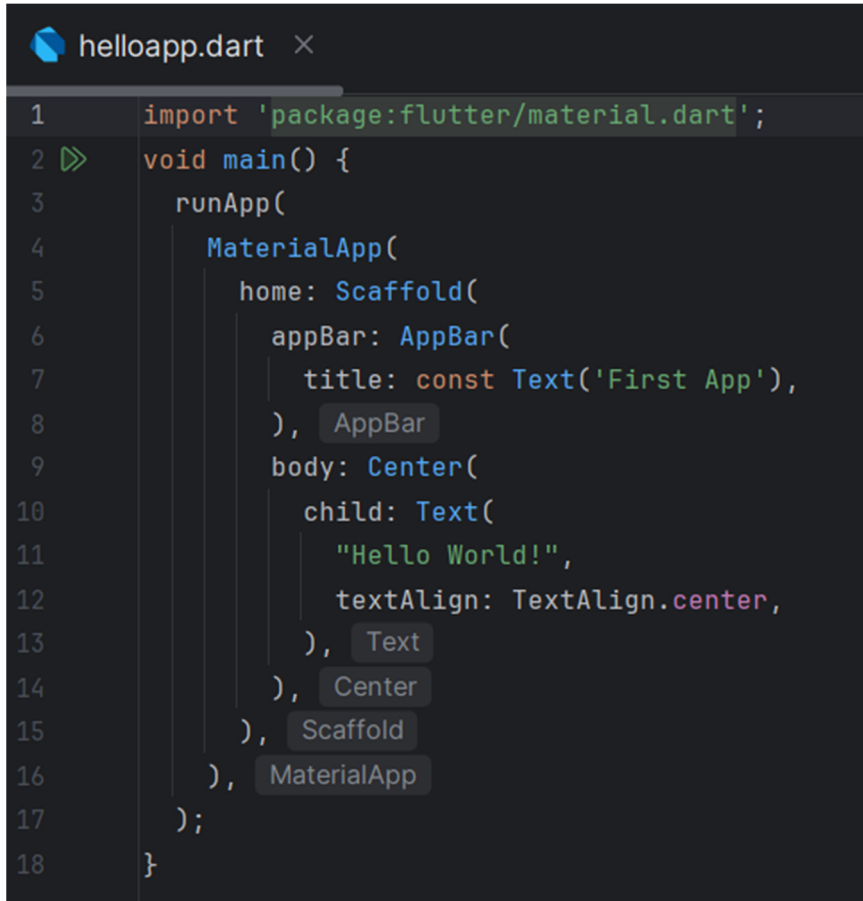
WEEK – 3

Create an application using Flutter to print hello world.

AIM: To create a simple application which display a message “Hello World”.

In the existing Flutter project, go to *File* → *New* → *Dart file* → enter name: “helloapp” (press enter)

PROGRAM [*helloapp.dart*]:

A screenshot of an IDE window titled 'helloapp.dart'. The code is as follows:

```
1 import 'package:flutter/material.dart';
2 void main() {
3   runApp(
4     MaterialApp(
5       home: Scaffold(
6         appBar: AppBar(
7           title: const Text('First App'),
8         ),
9         body: Center(
10          child: Text(
11            'Hello World!',
12            textAlign: TextAlign.center,
13          ),
14        ),
15      ),
16    ),
17  );
18 }
```

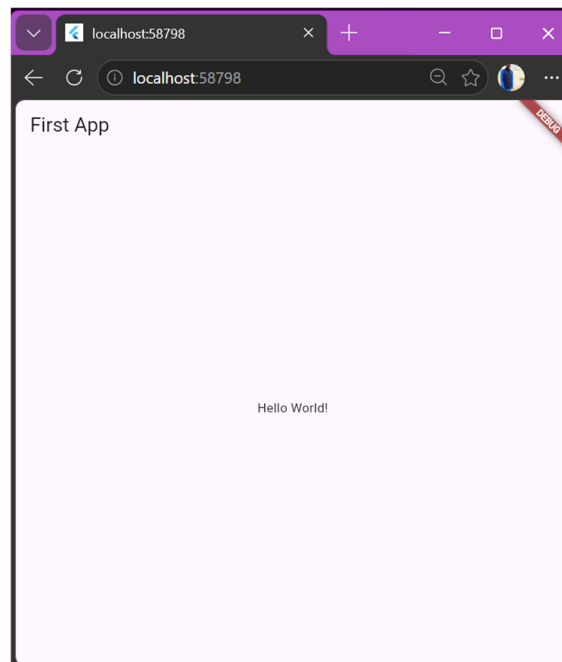
(or)

```
import 'package:flutter/material.dart';
void main() {
  runApp(
    MaterialApp(
      home: Scaffold(
        appBar: AppBar(
          title: const Text('First App'),
        ),
        body: Center(
          child: Text(
            'Hello World!',
            textAlign: TextAlign.center,
          ),
        ),
      ),
    ),
  );
}
```

How to Run: [This program file ‘helloapp.dart’ must be in “lib” folder, then only it executes.]

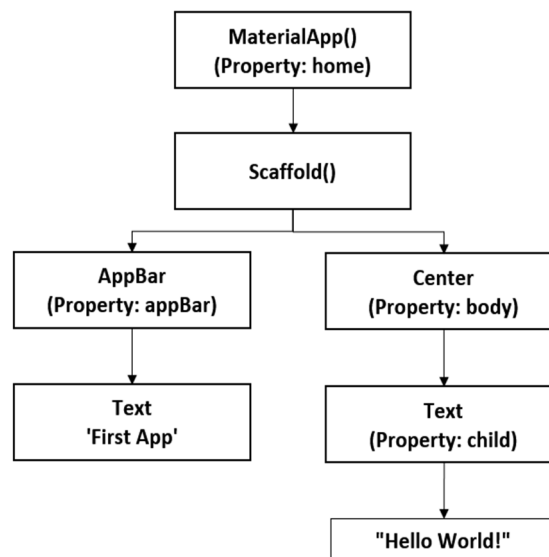
On the title bar, select where you want to display the output. Here we have chosen edge browser and click on run button (). Then the following is the result.

OUTPUT:



WIDGET TREE: Widgets are arranged in a hierarchical structure called a **Widget Tree**.

The Widget Tree for the above program is given as:



PROGRAM EXPLANATION:

1. `import 'package:flutter/material.dart';`

- The **import** keyword is used to include Flutter libraries.
- **material.dart** provides predefined classes and components required to design a Flutter application.

It contains:

- `MaterialApp`
- `Scaffold`
- `AppBar`
- `Center`
- `Text`

Without importing this package, Flutter UI components cannot be used.

2. `main()` Function: It is the starting point of every Dart application. The execution of the Flutter program starts from this function.

3. `runApp()` Function:

- It is the starting function of a Flutter application that takes the given widget and displays it on the device screen.
- It initializes the Flutter framework and attaches the root widget to the screen.

Why do we use `runApp()`?

- ⇒ To start the Flutter application.
 - ⇒ To load the first widget.
 - ⇒ To create the widget tree.
 - ⇒ To connect Flutter code with the device screen.
 - ⇒ To display the user interface.
-

4. `MaterialApp()`:

- `MaterialApp` provides the basic structure for a Flutter application.
- It follows Material Design principles.
- It acts as the **root widget** of an application and manages important features like:
 - Home screen
 - Themes
 - Navigation
 - Routes
 - Application title

5. home Property:

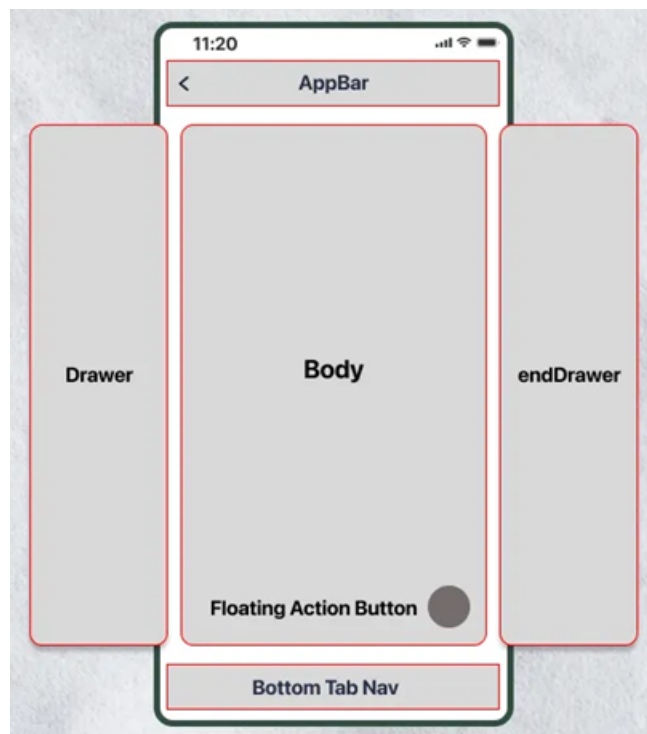
- The home property defines the **first screen** displayed when the app starts.
 - Here, Scaffold is the home screen.
-

6. Scaffold:

- It provides the **basic visual structure/layout of a screen** in an application.
- It implements the **Material Design layout structure** and provides predefined places for common screen components like:
 - AppBar
 - Body
 - Drawer
 - Floating Action Button
 - Bottom Navigation Bar
 - Bottom Sheet

It acts like a **container for arranging the main parts of an app screen**.

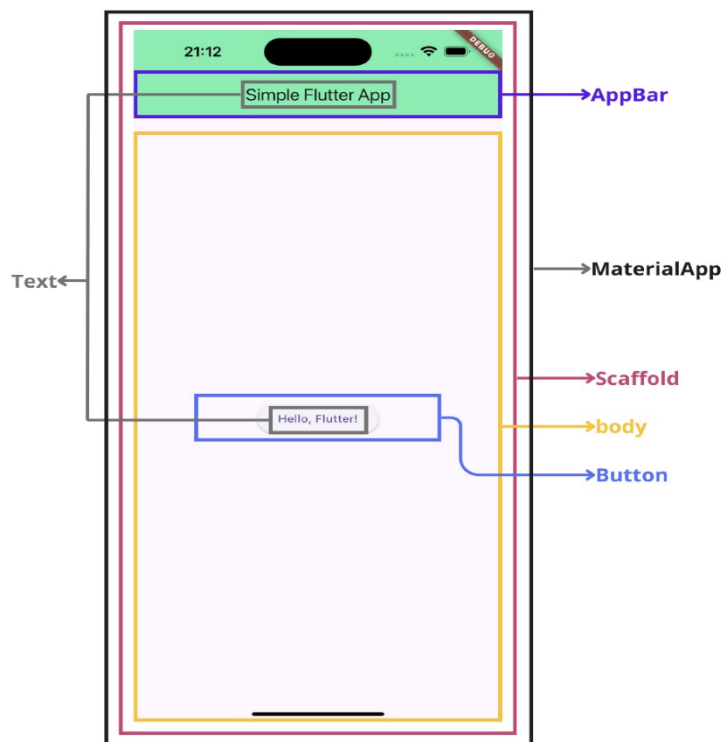
Scaffold Layout Design/Structure:



Scaffold Properties:

Property	Description
appBar	Displays the top application bar.
body	Contains the main content of the screen.
drawer	Creates a side navigation menu.
floatingActionButton	Creates a floating circular action button.
bottomNavigationBar	Displays navigation items at the bottom.
backgroundColor	Sets the screen background color.
bottomSheet	Displays information at the bottom of the screen.

Flutter Example Application:



7. Text Widget (AppBar Title):

- It displays text on the screen.
- Here it displays the title inside the AppBar.

8. child Property:

- **child** is used to place a widget inside another widget.
- It accepts only one widget.

WIDGETS

- A **widget** is an element of a graphical user interface that displays information or provides a specific way for a user to interact with the operating system or an application.
- A **widget** is a self-contained, mini-application or a tool designed to perform a specific function. Placing the widgets together creates the **widget tree**.
- For **example**, say an architect draws a blueprint of a house; all of the objects like walls, windows, and doors in the house are the widgets, and all of them work together to create the house or, in this case, the application.



Figure: Widgets on this Android phone provide information on time, date and weather.

WIDGETS IN FLUTTER:

- In Flutter, **widgets** are the basic building blocks used to create the user interface (UI) of an application. Everything displayed on the screen in Flutter is a widget.
- A widget describes how the UI should look and behave.
- **Examples:**
 - 1) Text displayed on screen
 - 2) Buttons
 - 3) Images
 - 4) Layouts
 - 5) Icons
 - 6) Complete application screens
- A Flutter application is a combination of many widgets arranged in a **widget tree**.

Types of Widgets in Flutter:

Flutter widgets are mainly divided into:

1. Stateless Widget

2. Stateful Widget